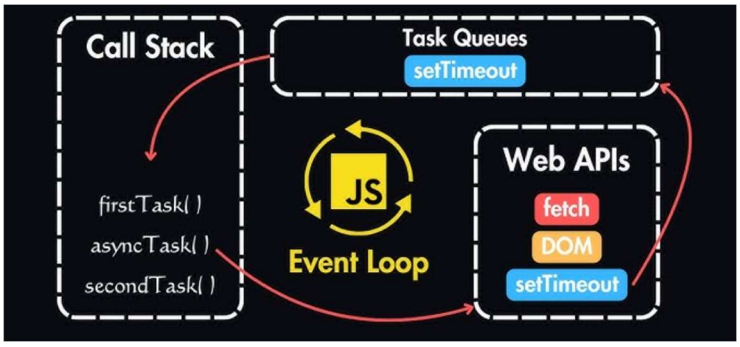
AI Driven
Consultancy

TypeScript Execution Model
Core Concepts

1.Single-Threaded Nature



What does Single-Threaded mean?

- TypeScript (via JS) uses ONE main thread
- Executes ONE operation at a time
- No parallel execution in the main thread

Why Single-Threaded?

- Simpler memory management
- Avoids race conditions
- Faster execution model

TypeScript Example

TS Executed line by line.ts U

src > Async Programming > TS Executed line by line.ts > ...

```typescript
let task1 = "Task 1";
let task2 = "Task 2";
let task3 = "Task 3";

console.log(task1);
console.log(task2);
console.log(task3);
// Output:
// Task 1
// Task 2
// Task 3
```

Note: Executed line by line, one after another.

AI Driven
Consultancy

2.Call Stack (How Functions Execute)

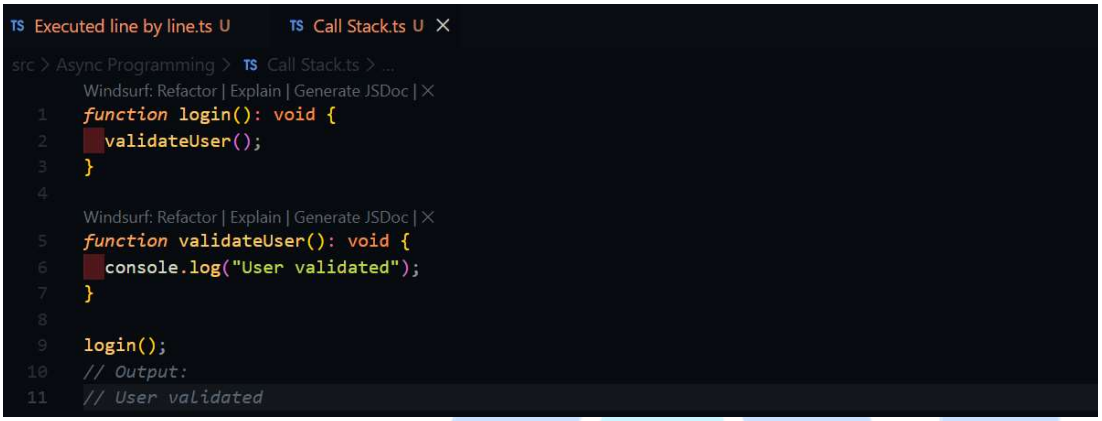
What is Call Stack?

- A stack data structure
- Tracks function execution
- Uses LIFO principle **(Last In, First Out)** rule

Why Call Stack is Needed?

- Keeps order of function calls
- Knows where to return after function finishes

TypeScript Example



Execution Flow

- login() → pushed to stack
- validateUser() → pushed to stack
- validateUser() completes → popped
- login() completes → popped

```
//Call Stack Visualization:

| validateUser() |  ← pushed
| login()        |  ← pushed

| login()        |  ← popped
|                |

|                |  ← stack empty
```

Note: Stack becomes empty → program continues.

3. Web APIs (Node.js Runtime Helpers)

What are Web APIs?

Web APIs are provided by the **runtime environment** (**Node.js** or **Browser**), not by TypeScript itself.

They are used to handle time-consuming or non-blocking tasks, such as:

- Timers
- File operations
- Network requests
- Database queries

TypeScript/JavaScript only requests these operations; the actual work is done outside the Call Stack.

Why Web APIs Are Needed

JavaScript is single-threaded, meaning it can execute only one task at a time.

If slow operations ran inside the Call Stack:

- The program would freeze
- The UI or server would hang
- No other code could run

Web APIs prevent this by executing slow tasks outside the Call Stack.

TypeScript Example

AI Driven
Consultancy

Execution Flow

Step 1: console.log("Start")

- Added to the Call Stack
- Executed immediately

Step 2: setTimeout (...)

- Timer is registered
- Handed over to the Node.js Web API
- Call Stack becomes free immediately

The timer runs **outside JavaScript execution**.

Step 3: console.log("End")

- Executed immediately
- Does not wait for the timer

Step 4: After 2 seconds

- Timer completes in the Web API
- Callback is placed into the Callback Queue
- Event Loop checks if the Call Stack is empty
- Callback is pushed to the Call Stack and executed

```
console.log("Timer finished");
```

Output

```
// Output:

    // Start
    // End
    // Timer finished
```

The timer output appears last because it is handled by the Web API, not the Call Stack.

One-Line Summary

Web APIs allow JavaScript to run asynchronously by moving slow operations outside the Call Stack.

4. Event Loop

What is an Event Loop?

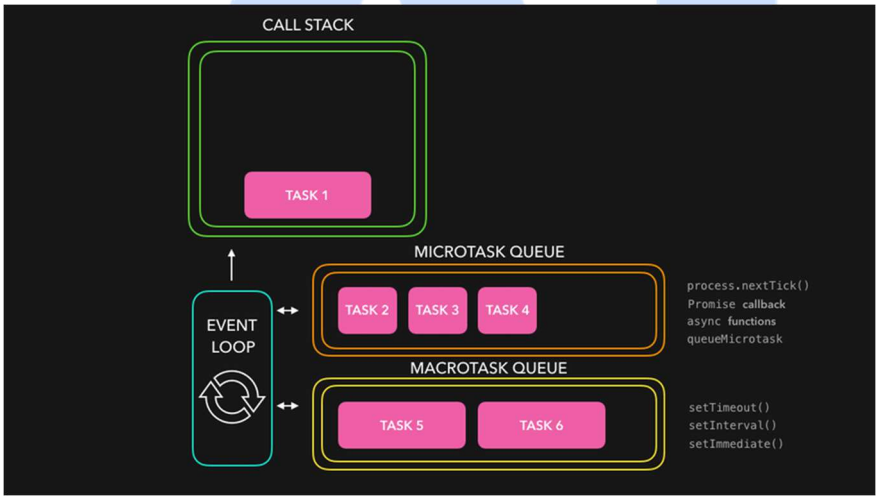
The ***Event Loop*** is a ***background system*** in ***JavaScript*** that ***decides*** what ***code should run next***.

It works like this:

- JavaScript can do only one task at a time
- Other tasks (like timers, clicks, network calls) must wait
- The Event Loop manages this waiting and running

How the Event Loop Works

1. JavaScript runs the current task
2. Other tasks wait in a line (queue)
3. When the current task finishes:
    - Event Loop picks the next task
    - Runs it
4. This continues until no tasks are left



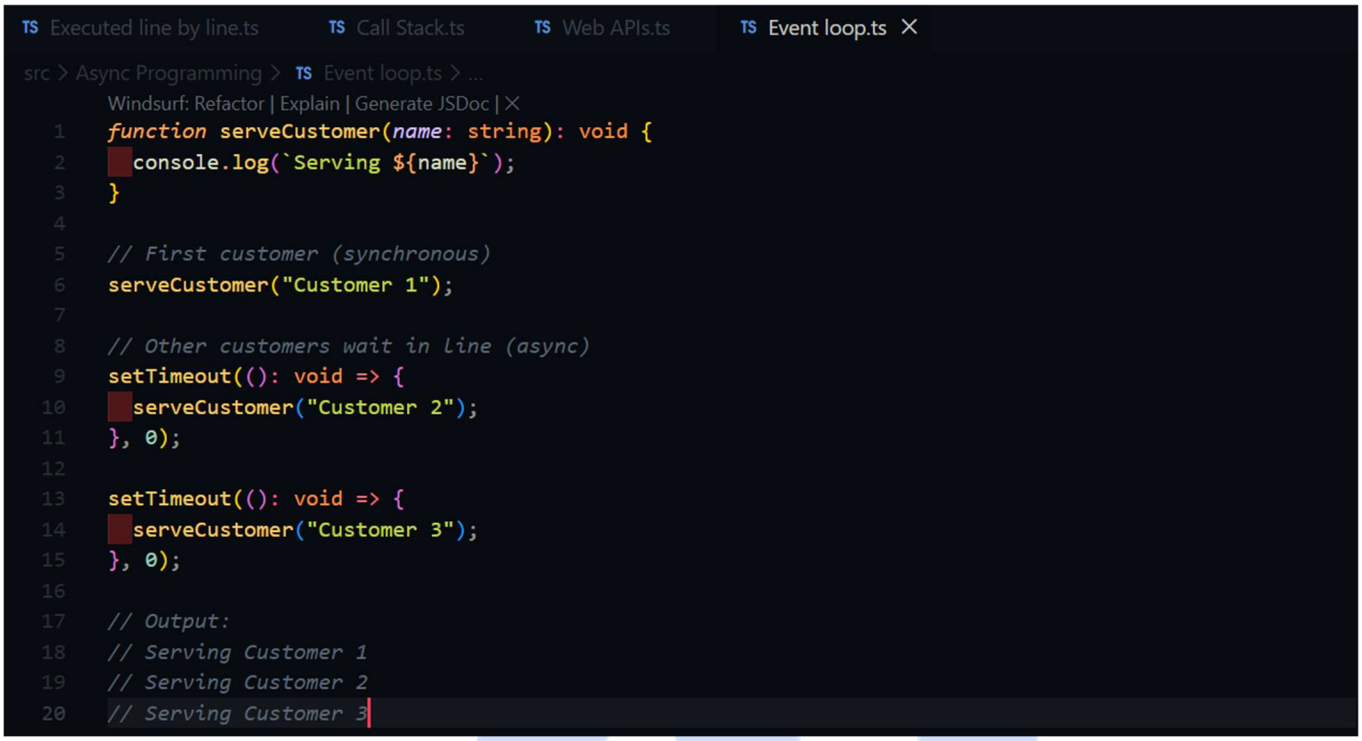
Why the Event Loop is Needed?

Without the Event Loop	With the Event Loop
• The browser would freeze • The page would stop responding • Servers would get blocked	• JavaScript stays fast • Multiple events are handled smoothly • No freezing or blocking

One-Line Simple Definition

The Event Loop lets JavaScript finish one task, then safely run the next waiting task without freezing the app.

TypeScript Example



Execution Flow

Step 1: Customer 1

```
// First customer (synchronous)
serveCustomer("Customer 1");
```

- Cashier serves Customer 1 immediately
- This is **JavaScript running now**

```
// Output:
// Serving Customer 1
```

Step 2: Customer 2 and 3 join the line

```
setTimeout(): void => {
serveCustomer("Customer 2");
```

And

```
setTimeout(): void => {
serveCustomer("Customer 3");
```

- These customers are put in the **queue**
- Cashier does not serve them yet
- They are waiting

Step 3: Event Loop checks

- Is cashier free? Yes
- Take next customer from line

Step 4: Serve waiting customers

```
setTimeout(():  void => {
    serveCustomer("Customer 2");
```

```
setTimeout(():  void => {
    serveCustomer("Customer 3");
```

Output

```
// Output:
// Serving Customer 1
// Serving Customer 2
// Serving Customer 3
```

Easy Real-Life Example

Think of a **single cashier** at a shop:

- One customer is served at a time
- Other customers wait in a line
- When one finishes, the next is served

The cashier ---------- JavaScript
The line ---------- Queue
Choosing the next customer ----------Event Loop

One-Line Memory Tip

JavaScript serves one task at a time; Event Loop picks the next waiting task.